

Basic Concepts in Programming

Fundamentals of Programming

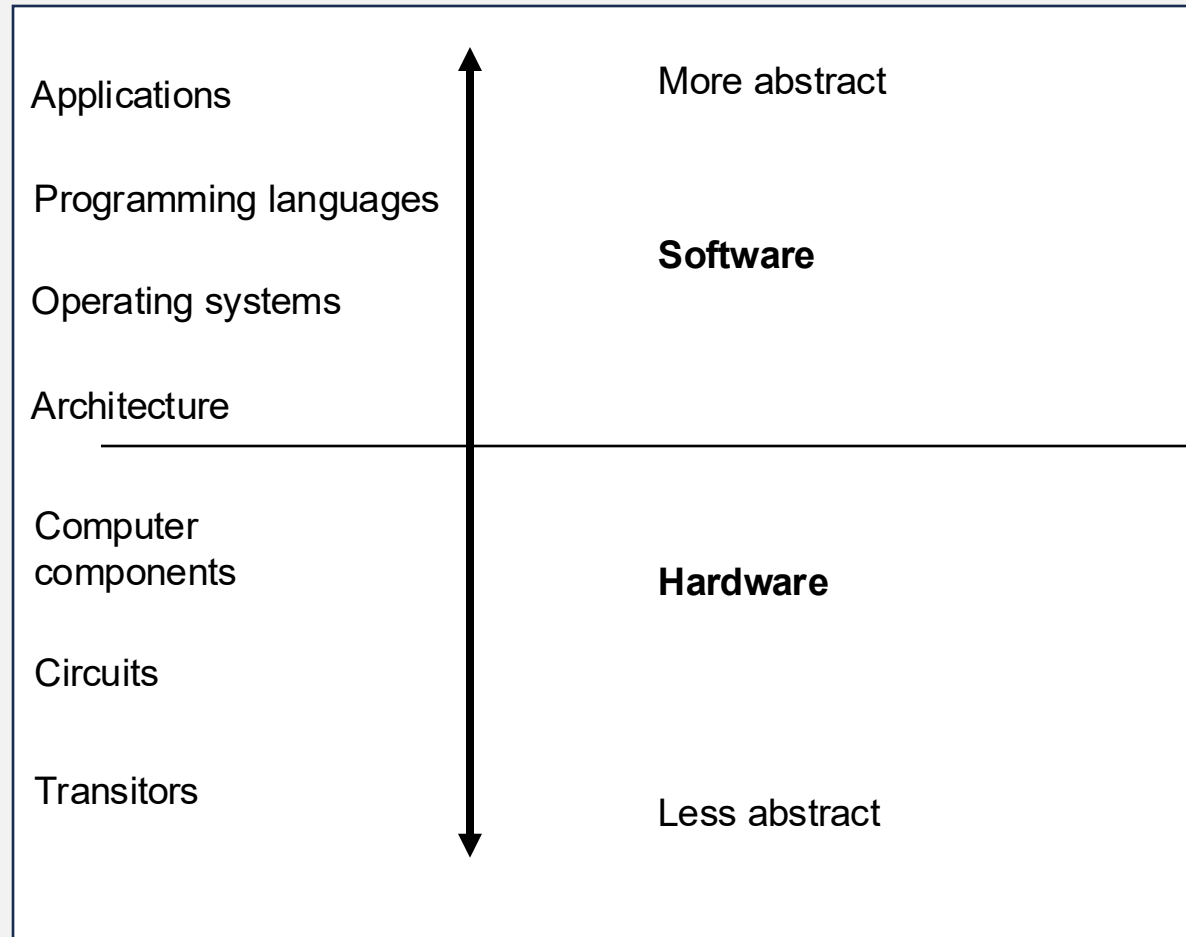
Bùi Duy Đăng

bddang@fit.hcmus.edu.vn

Plan for today

- Review
- Basic concepts in programming
- Exercises

Review



Compiler and Interpreter

- Pros and Cons
- Which languages support compilers?
- Which languages support interpreter?

Bugs/debugs

- Syntax and semantic errors
- When does compile-time error occur?
- When does run-time error occur?

Programming language

- A language to communicate with a computer
- Low-level programming language
 - Machine code, Assembly
- High-level programming language
 - Procedural: Fortran, Pascal, C
 - Functional: Lisp, Haskell, F#
 - Object oriented: C++, Java, C#
- Others
 - Scripting languages: JavaScript, Python
 - AI prompts: ask AI to generate source code

Algorithm

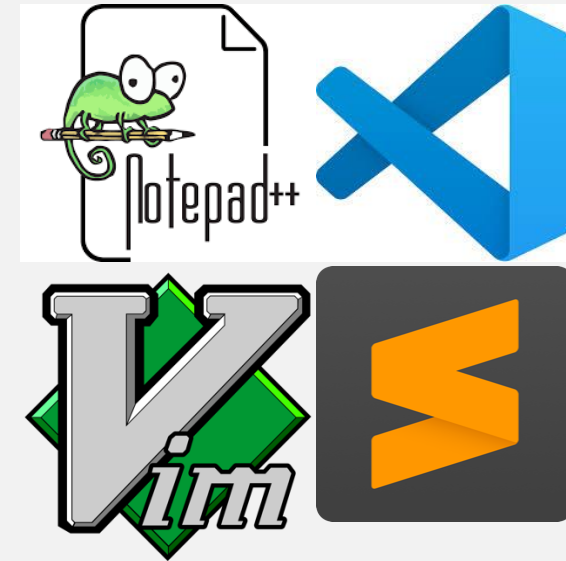
A finite sequence of steps to solve a problem

- Given a year, check whether it is a leap year or not?
- Given a number, check whether it is a prime or not?

Basic concepts in programming

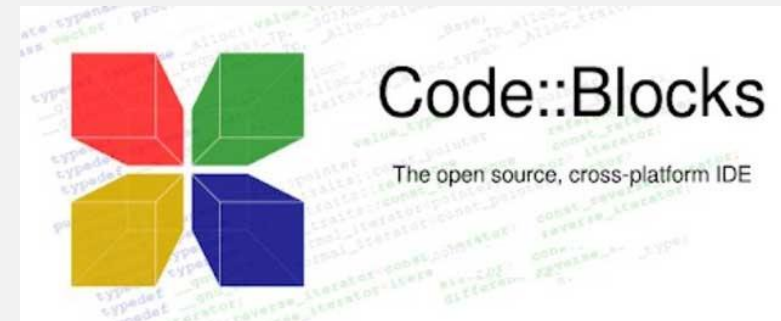
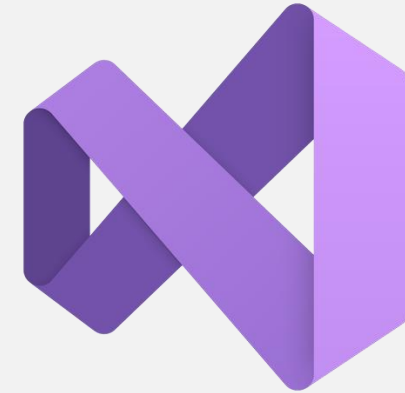
Programming environment

- A platform of tools supporting programming tasks
- Source code editor
 - Edit text, syntax highlight, code completion
- Compiler
 - Translate source code to machine code
- Terminal
 - Command-line tool to run program and shell commands
- Debugger
 - Run program step-by-step
 - Monitor computer memory



Programming enviroment

- IDE – Integrated Development Environment
 - “All-in-one” tools
 - User friendly, configuration-free, add-in features
 - Cons:
 - Heavy memory and storage usage
 - Limit configuration skill
 - Require license
 - Track user data



Programming enviroment

Tools	Windows	MacOS	Linux
Editor	Notepad++ VS Code (*) Sublime Text (*)	TextMate VS Code (*) Sublime Text (*)	Vim, Emacs VS Code (*) Sublime Text (*)
Compiler	MSVC (*) GCC (+) Clang (+)	Clang GCC	GCC Clang
Debugger	MSVC (*) GDB	LLDB GDB	GDB LLDB
IDE	Visual Studio (*) Code::Blocks CodeLite	Xcode (*) Code::Blocks CodeLite	Code::Blocks CodeLite

(*): Proprietary software and track user data.

(+): Run on Linux simulated environment like MinGW64, WSL.

Comments

Comments

- **Comments** are code lines to refer comments of programmers
 - All languages support
- It is very necessary for programmers
 - Review
 - Others read
- Reading a piece of comments instead of reading whole code
- For example: [1](#), [2](#)

Comments in C++

- Line comments

// this is a comment until the end line

- Block comments

/* this is the first comment in a block
this comment will be stop until seeing */

```
#if 0  
#endif
```

Variables

Variables

- **Variables** are names/labels to label data in memory
 - Used to **store** values
- **Variable names** are case-sensitive,
 - Capital and lowercase letters are different (e.g., "hello" and "Hello")
 - Hungarian, Camel, snake style
- Contains only alphabetic letters, underscores ("_") or numbers
 - MUST not start with a number
- Cannot be keywords in computer (e.g., if, for)

Meaningful variable names are essential

Constant Variables

- **Constant variables** are names/labels to label data in memory
 - Used to **store unchangeable** values
- In C++,
 - Names of constant variables are often written in uppercase (e.g., PI, MAX, CURRENTYEAR)

Data types

Data Types

- Computer stores **data** digitally in memories
 - **A classification of data** which tells the compiler or interpreter how the programmer intends to use the data
- Different data types:
 - Simple, built-in data types (e.g., integer, double)
 - User-defined (e.g., struct, class)
 - Others (e.g., pointer in C++)

Boolean

- Applied to "things" that have two values
 - ON/OFF, open/close, true/false
- It contains **true** or **false**
- In C++, denoted as `bool`, in an indirect way
 - `false`: value 0
 - `true`: value not 0
- For example:
 - 0 (`false`), 1 (`true`), -1 (`true`), 2 (`true`)
 - 1 < 2 (`true`), 3 > 1 (`true`)

Character

- Indicate **each** character value in computer
- In C/C++,

Name: **char**

Range: 256 characters in ASCII Table

Integers can be used to represent

- Unicode, UTF-8

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Integer

- Indicate negative, zero, and non-negative numbers in mathematics in C++
 - Signed integers
 - Unsigned integers

Data Type	Typical Size	Typical Range
<code>short int</code>	2 bytes	−32,768 to +32,767
<code>unsigned short int</code>	2 bytes	0 to +65,535
<code>int</code>	4 bytes	−2,147,483,648 to +2,147,483,647
<code>unsigned int</code>	4 bytes	0 to 4,294,967,295
<code>long int</code>	4 bytes	−2,147,483,648 to +2,147,483,647
<code>unsigned long int</code>	4 bytes	0 to 4,294,967,295
<code>long long int</code>	8 bytes	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
<code>unsigned long long int</code>	8 bytes	0 to 18,446,744,073,709,551,615

Real Numbers

- Contains integers, rational, and irrational numbers in mathematics
- In C++, using floating-point to store
 - `float`: 4 bytes, precision up to 7 digits
 - `double`: 8 bytes, precision up to 15 digits

Floating Point Data Types			
Type	Size(in bytes)	Range	Digits of Precision
float	4	$3.4 \cdot 10^{-38}$ to $3.4 \cdot 10^{38}$	7
double	8	$1.7 \cdot 10^{-308}$ to $1.7 \cdot 10^{308}$	15

Arithmetic Operators

- Pre-defined in programming languages for data types
 - In C++,
 - Addition +
 - Subtraction −
 - Multiplication *
 - Division /
 - Modulus %
- Which operators used for integer and/or real numbers?
- Unary, binary, and ternary operators

Unary Operators

- Only have one operand in a expression
- In C++,
 - `++` (increment, one value), `--` (decrement, one value)
 - Put before operand
E.g., `++x` or `--x`: do increment/decrement **first**.
 - Put after operand
E.g., `x++` or `x--`: do increment/decrement **after**.

```
x = 10; y = x++;           // y = 10 and x = 11
x = 10; y = ++x;           // x = 11 and y = 11
```

Bitwise Operators

- Used for bit of operands (integers)
- In C++,
 - and **&**, or **|**, xor **^**, not **~**
 - shift right **>>**, shift left **<<**
 - Compound assignment operators: **&=**, **|=**, **^=**, **~=**, **>>=**, **<<=**

&	0	1
0	0	0
1	0	1

^	0	1
0	0	1
1	1	0

 	0	1
0	0	1
1	1	1

~	0	1
	1	0

Relational Operators

- Check the relationship between two operands

- Return **true** or **false**

- In C++,

- **==**, **!=**, **>**, **<**, **>=**, **<=**

- For example:

`s1 = (1 == 2) ; s3 = (1 > 2) ; s5 = (1 < 2) ;`

`s2 = (1 != 2) ; s4 = (1 >= 2) ; s6 = (1 <= 2) ;`

Logical Operators

- Check whether an expression is **true** or **false**

- In C++,

and **& &**,

or **| |**,

not **!**

- For example

s1 = (1 > 2) **& &** (3 > 4) ;

s2 = (1 > 2) **| |** (3 > 4) ;

s3 = **!** (1 > 2) ;

&&	0	1
0	0	0
1	0	1
 	0	1
0	0	1
1	1	1

C++ Operator Precedence

Precedence	Operator	Description	Associativity
1	::	Scope resolution	Left-to-right →
2	a++ a-- type() type{} a() a[] . ->	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access	
3	++a --a +a -a ! ~ (type) *a &a sizeof co_await new new[] delete delete[]	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of ^[note 1] await-expression (C++20) Dynamic memory allocation Dynamic memory deallocation	Right-to-left ←

Source: https://en.cppreference.com/w/cpp/language/operator_precedence

C++ Operator Precedence

4	. * ->*	Pointer-to-member	Left-to-right →
5	a*b a/b a%b	Multiplication, division, and remainder	
6	a+b a-b	Addition and subtraction	
7	<< >>	Bitwise left shift and right shift	
8	<=>	Three-way comparison operator (since C++20)	
9	< <= > >=	For relational operators < and ≤ and > and ≥ respectively	
10	== !=	For equality operators = and ≠ respectively	
11	a&b	Bitwise AND	
12	^	Bitwise XOR (exclusive or)	
13		Bitwise OR (inclusive or)	
14	&&	Logical AND	
15		Logical OR	
16	a?b:c	Ternary conditional ^[note 2]	Right-to-left ←
	throw	throw operator	
	co_yield	yield-expression (C++20)	
	=	Direct assignment (provided by default for C++ classes)	
	+= -=	Compound assignment by sum and difference	
	*= /= %=	Compound assignment by product, quotient, and remainder	
17	<<= >>=	Compound assignment by bitwise left shift and right shift	Left-to-right →
	&= ^= =	Compound assignment by bitwise AND, XOR, and OR	
	,	Comma	

Variable Declaration

DataType *Name*;

For example,

```
int a;
```

```
double x, y, z;
```


Constant Variables

- Memory location whose content **cannot change** during execution.

- Syntax

```
const DataType VariableName;
```

- For example

```
const double PI = 3.14;  
const int NO_OF_STUDENTS = 60;
```

Assignments operators

Variable = *expression*;

Expressions can be **values**, **variables**, or **complex expressions**

- In C++,

```
int x = 0;    // initialization, equivalent to int x(0);  
float y;      // what is the value of y?  
y = 1.5;  
y = x;  
y = (3 + x) * 4;
```

Statements

- Instructs the computer to do a specific task
- In C++, compiler **ignores space** (blank, tab, enter) in a statement
- **Ends with a semicolon (;)**
- For example

`a=292;`

`a = 292;`

`a`

`=`

`292;`

Statements

- Single statement
- Compound statement: a group of statements enclosed in curly brace { and }

For example:

```
a = 2912;           // Single statement
```

```
{                   // Compound statement
```

```
    a = 2912;
```

```
    b = 1706;
```

```
}
```

Output statement

- In C++,

```
#include <iostream>    // declare iostream library
// using namespace std;    // comment
int main() {
    // std::cout << expression;
    // this is an output statement example
    std::cout << "hello world!";

    return 0;
}
```

Output statement

- Escape sequence

```
cout << "hello world!\n" // Equivalent to  
cout << "hello world!" << endl; // endl
```

	Escape Sequence	Description
\n	Newline	Cursor moves to the beginning of the next line
\t	Tab	Cursor moves to the next tab stop
\b	Backspace	Cursor moves one space to the left
\r	Return	Cursor moves to the beginning of the current line (not the next line)
\\	Backslash	Backslash is printed
\'	Single quotation	Single quotation mark is printed
\"	Double quotation	Double quotation mark is printed

Input statement

- In C++,

```
std::cin >> variable ;
```

```
#include <iostream>
```

```
// using namespace std;
```

```
...
```

```
int yearBorn;      // input the year born,
```

```
int currentYear = 2025;
```

```
std::cin >> yearBorn; // suppose a valid year
```

```
std::cout << "your age is " << currentYear -  
yearBorn << endl;
```

Some useful functions

- Math functions
- `#include <cmath> // math library`
 - `pow(x, y)`
 - `acos(x), asin(x), atan(x), cos(x), sin(x), ...`
 - `exp(x), log(x), log10(x)`
 - `sqrt(x)`
 - `ceil(x), floor(x)`
 - `abs(x), fabs(x)`

Exercises

- 1) Print out some characters in ASCII Table in console and list some characters you interested.
- 2) Print out some information and escape sequence
E.g.,: *Student_name*\n
- 3) Given a number year, calculate its age
- 4) Convert speed value to pace value and vise versa
- 5) Convert metter value to mile value and vise versa

Exercises

- 6) Given a *year*, check if is a leap year.
- 7) Given scores of three courses and its coefficients. Calculate the average score.
- 8) Given an *average score*, display rank of this score, such as excellent (9-10), very good (8-9), good (7-8), fair (5-7), poor (<5)
- 9) Given three average scores of students, display ranking of three students. For example: Input: 8 9 10. Output: 1 2 3
- 10) Given a number *n*, calculate $1 + 2 + \dots + n$